PART IIB REVISION NOTES

4F12: Computer Vision

Qianshuo (Harry) Ye
April 2026

1 Feature Detection and Matching

Image structure:

- *Featureless region:* smooth variation of intensities.
- *Edge:* intensity discontinuity in one direction.
- *Corner:* intensity discontinuity in two directions.

1D edge detection: The general procedure is:

1. Convolve the signal $I(x)$ with a Gaussian kernel $g_\sigma(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{x^2}{2\sigma^2}\right)$.
2. Compute the derivative $s'(x) = \frac{d}{dx}[I(x) * g_\sigma(x)]$.
3. Find maxima and minima of $s'(x)$.
4. Use thresholding to mark edges.

To save computation, directly find zero crossings of $s''(x)$, obtained by convolving the signal with the *Laplacian* of a Gaussian

$$s''(x) = g''_\sigma(x) * I(x)$$

The parameter σ controls the scale of edge detection: larger σ detects coarser edges.

2D edge detection: The general procedure is similar to the 1D case, but the image is smoothed with a 2D Gaussian kernel

$$G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

Canny detector: The gradient of the smoothed image ∇S is used to perform non-maximum suppresion: place edge elements (edgels) where $|\nabla S|$ is greater than local values of $|\nabla S|$ along the direction of $\pm\nabla S$. The edgels are thresholded – only those with $|\nabla S|$ above a certain value are kept.

Marr-Hildreth operator: Find the zero crossings of $\nabla^2 G_\sigma * I$, where $\nabla^2 G_\sigma$ is the Laplacian of the G_σ.

In practice, the convolution are performed as truncated summations (typically the discarded samples are $< 1/1000$ of the peak value)

$$S(x, y) = \sum_{u=-n}^{n} \sum_{v=-n}^{n} G_\sigma(u, v) I(x - u, y - v)$$

The 2D convolution can be decomposed into two 1D convolutions

$$S(x,y) = g_\sigma(x) * [g_\sigma(y) * I(x,y)]$$

The computational saving is $\frac{(2n+1)^2}{2(2n+1)} = \frac{2n+1}{2}$.

Differentiation is also implemetned through a discrete concolution with the kenerl $[1/2, 0, -1/2]$, using the approximation $\frac{\partial S}{\partial x} \approx \frac{S(x+1,y)-S(x-1,y)}{2}$.

Corner detection: One way is to compute the normalised cross-correlation between an image patch P and other portions of the image I:

$$c(x,y) = \frac{\sum_{u=-n}^{n}\sum_{v=-n}^{n} I(x+u,y+v)P(u,v)}{\sqrt{\sum_{u=-n}^{n}\sum_{v=-n}^{n} I(x+u,y+v)^2 \sum_{u=-n}^{n}\sum_{v=-n}^{n} P(u,v)^2}}$$

A patch with a well-defined peak in its correlation function is a corner.

A more efficient method is to use the intensity difference:

1. Calculate change in intensity in direction $\mathbf{n}$:

$$S_n = \nabla S(x,y) \cdot \hat{\mathbf{n}} = [S_x \quad S_y]^\top \cdot \hat{\mathbf{n}}$$
$$S_n^2 = \frac{\mathbf{n}^\top \begin{bmatrix} S_x^2 & S_xS_y \\ S_xS_y & S_y^2 \end{bmatrix} \mathbf{n}}{\mathbf{n}^\top \mathbf{n}}$$

2. Smooth S_n^2 with a Gaussian kernel:

$$C_n(x,y) = G_\sigma * S_n^2 = \frac{\mathbf{n}^\top \overbrace{\begin{bmatrix} \langle S_x^2 \rangle & \langle S_xS_y \rangle \\ \langle S_xS_y \rangle & \langle S_y^2 \rangle \end{bmatrix}}^{A} \mathbf{n}}{\mathbf{n}^\top \mathbf{n}}$$

   where $\langle \cdot \rangle$ denotes the smoothed value.

3. According to eigenvector theory, $\lambda_1 \leq C_n(x,y) \leq \lambda_2$. λ_1 and λ_2 are eigenvalues of A. λ_2 is the maximum smoothed change in intensity and λ_1 is the minimum. A corner is detected if λ_1 and λ_2 are both large and distinct.

The Harris corner detector computes the determinant $\det(A) = \lambda_1\lambda_2$ and the trace $\mathrm{tr}(A) = \lambda_1 + \lambda_2$ to avoid actual computing eigenvalues. A corner is detected when $\lambda_1\lambda_2 - \kappa(\lambda_1 + \lambda_2)^2$ is greater than some threshold.

Blob detection: A *blob* is an area of uniform intensity. Convolution with the Laplacian of Gaussian (LoG) kernel produce a strong response with an extrama at the center of a blob when the scale of the kernel σ matches the size of the blob. The radius of the matched blob is $\sqrt{2}\sigma$ (where zero-crossing occurs).

Image pyramids: A sequence of images smootheed with Gaussians of different scales. The scale satisfies $\sigma_{i+1} = 2^{1/s}\sigma_i$ so it doubles ever s levels. After s levels, double the scale by subsampling the image.

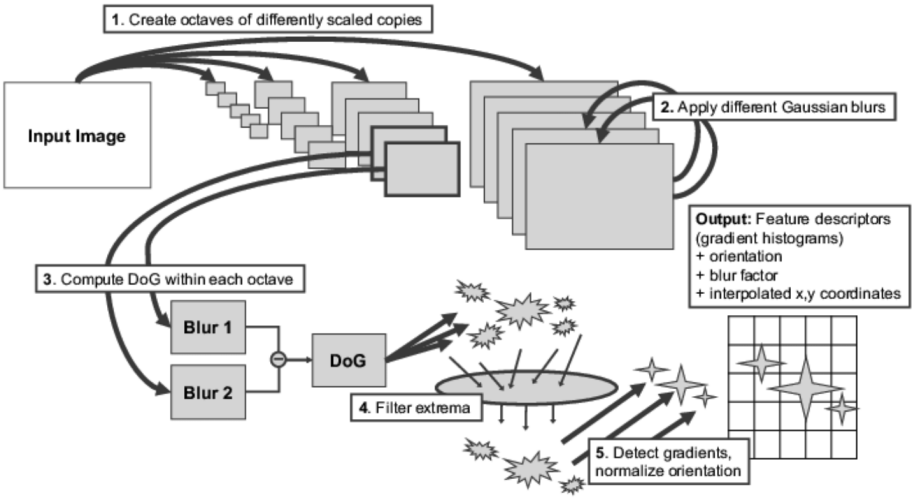
The image is smoothed incrementally by convolution $G(\sigma_{i+1}) = G(\sigma_i) * G(\sigma_{k_i})$, where $\sigma_{k_i} = \sqrt{\sigma_{i+1}^2 - \sigma_i^2} = \sigma_i\sqrt{2^{2/s} - 1}$ (only need to be computed once).

Difference of Gaussian (DoG): A blob detector that approximates the LoG:

$$G(x, y, k\sigma) - G(x, y, \sigma) \approx (k-1)\sigma^2 \nabla^2 G(x, y, \sigma)$$

Keypoint detection: Blob centeres are found by subtracting neighbouring images of same dimension in the image pyramid and search for local extrema (by comparing with 26 neighbouring pixels in the current, upper and lower levels). Normalise for scale of the keypoint by sampling neighbouring pixels in the appropriate image of the Image pyramid.

Keypoint orientation: Build a histogram (36 bins covering 360 degrees) of edge orientations weighted by their gradient magnitudes in a 16×16 neighbourhood. The gradients needs to be smoothed by a 2D Gaussian of size $1.5\times$ the blob scale. The highest peak in the histogram is the dominant orientation. Normalise for orientation by sampling neighbouring pixels after computing a dominant orientation.



Matching patches: Patches of intensities can be matched by the cross-correlation:

$$CC(P_1, P_2) = \sum_i^N P_1[i] P_2[i]$$

The patch can be normalised to make the matching invariant to changes in brightness and contrast:

$$\mu = \frac{1}{N} \sum_{x,y} I(x, y), \quad Z(x, y) = I(x, y) - \mu$$

$$\sigma^2 = \frac{1}{N} \sum_{x,y} Z(x, y)^2, \quad P(x, y) = \frac{Z(x, y)}{\sigma}$$

Scale-Invariant Feature Transform (SIFT):

1. Split $N \times N$ ppatch into c cells, calculate gradient at every pixel
2. For each cell, build a histogram of gradient directions (d directions) weighted by their magnitude and a Gaussian window with a σ of 0.5 times the feature scale (to avoid occlusion).

3. Normalise the descriptor to make it invariant to changes in gradient magnitude.
4. Threshold the maximum value of the descriptor to 0.2 to reduce the effect of storng lights, then renormalise.

Typically, $N = 16$, $c = 16$ and $d = 8$, so the SIFT descriptor is a 128-dimensional vector.

A match can be measured by the Euclidean distance between two SIFT descriptors (linear search):

$$E(\vec{x}, \vec{y}) = \sqrt{\sum_d (x_d - y_d)^2}$$

An alternative is to use a k-d tree to speed up the search.

Texture: Orderly and repetitive patterns of regular sub-elements called *textons*. Textures can be described by their response to a collection of filters.

2 Projection

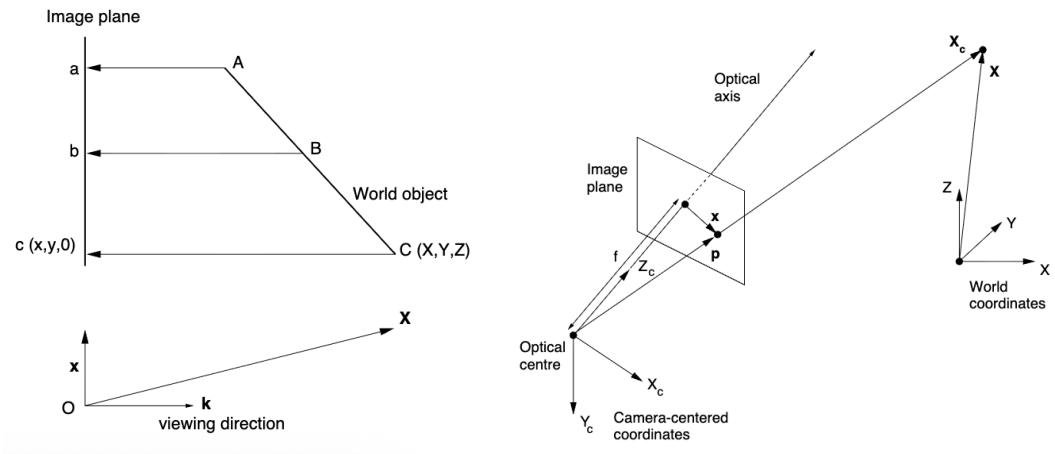
Orthographic projection: Project the scene onto an image plan using parallel rays. The projection is given by

$$\mathbf{x} = \mathbf{X} - (\mathbf{X} \cdot \mathbf{k})\mathbf{k} = (\mathbf{k} \times \mathbf{X}) \times \mathbf{k}$$

Perspective projection: Project the scene onto an image plane using rays that converge at a single point (the camera center). The projection is given by

$$\mathbf{x} = \frac{f}{Z_c} \begin{bmatrix} X_c \\ Y_c \end{bmatrix}$$

where f is the focal length and Z_c is along the optical axis.



Vanishing points: Points where parallel lines in the world meet in the image. Parallel plane meet in a line in the image called the *horizon line*. Any set of parallel lines on the planes will have a vanishing point on the horizon line. Vertical lines have a vanishing point at infinity.

Full camera model: A full camera describes the mapping fro world to pixel coordinates, consisting of 3 transformations:

1. Rigid body motion between camera and the scene – from world coordinates to camera coordinates:

$$\mathbf{X}_c = \mathrm{R}\mathbf{X} + \mathbf{T} \text{ (rotation and translation)}$$

2. Perspective projection – mapping 3D points to 2D image plane:

$$x = \frac{fX_c}{Z_c}, \quad y = \frac{fY_c}{Z_c}$$

3. CCD imaging – mapping the image plane to pixel coordinates:

$$u = u_0 + k_u x, \quad v = v_0 + k_v y$$

Homogeneous coordinates: represent each point in the image plane by a 3D vector and each point in the world by a 4D vector:

$$\tilde{\mathbf{X}}_c = \begin{bmatrix} \lambda X_c \\ \lambda Y_c \\ \lambda Z_c \\ \lambda \end{bmatrix} \quad \tilde{\mathbf{x}} = \begin{bmatrix} sx \\ sy \\ s \end{bmatrix}$$

The homogeneous equations are

$$a_1 X_c + a_2 Y_c + a_3 Z_c + a_4 = 0 \quad b_1 x + b_2 y + b_3 = 0$$

Points are at infinity when $\lambda = 0$ or $s = 0$. $\tilde{\mathbf{X}} = \mathbf{0}$ is undefined.

Perspective projection can be expressed as:

$$\tilde{\mathbf{x}} = \begin{bmatrix} sx \\ sy \\ s \end{bmatrix} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} \lambda X_c \\ \lambda Y_c \\ \lambda Z_c \\ \lambda \end{bmatrix} = \mathrm{P}_p \tilde{\mathbf{X}}_c$$

Camera projection matrix: The full camera model can be expressed as

$$\tilde{w} = \mathrm{P}_{ps}\tilde{\mathbf{X}} = \mathrm{P}_c \mathrm{P}_p \mathrm{P}_r \tilde{\mathbf{X}}$$

$$\text{where } \mathrm{P}_r = \begin{bmatrix} \mathrm{R} & \mathbf{T} \\ \mathbf{0}^\top & 1 \end{bmatrix} \quad \mathrm{P}_p = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad \mathrm{P}_c = \begin{bmatrix} k_u & 0 & u_0 \\ 0 & k_v & v_0 \\ 0 & 0 & 1 \end{bmatrix}$$

The projection matrix can be decomposed into a *camera calibration matrix* $\mathbf{K}$ and a rigid body motion $[R|\mathbf{T}]$: $\mathrm{P}_{ps} = \mathbf{K}[R|\mathbf{T}]$, where

$$\mathbf{K} = \begin{bmatrix} fk_u & 0 & u_0 \\ 0 & fk_v & v_0 \\ 0 & 0 & 1 \end{bmatrix}$$

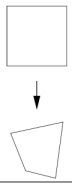
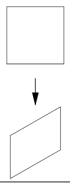
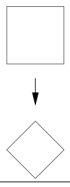
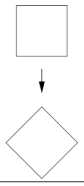
The ratio fk_v/fk_u is the *aspect ratio*.

Projective camera: A more general camera model, described by a 3×4 matrix P.

When viewing a plane, the projection is a *homography* that can be described by a 3×3 matrix. When viewing a line, the projection can be described by a 3×2 matrix.

The degree of freedom of the matrices are $n - 1$ as the scale factor is arbitrary.

Planar transformation: Mapping between two planes.

Group	Matrix	Distortion	Invariants
projective 8 DOF	$\begin{bmatrix} p_{11} & p_{12} & p_{13} \\ p_{21} & p_{22} & p_{23} \\ p_{31} & p_{32} & p_{33} \end{bmatrix}$		concurrency and collinearity, order of contact, tangent discontinuities and cusps, cross-ratio of four collinear points, measurements in canonical view
affine 6 DOF	$\begin{bmatrix} p_{11} & p_{12} & p_{13} \\ p_{21} & p_{22} & p_{23} \\ 0 & 0 & p_{33} \end{bmatrix}$		all the above, plus parallelism, ratio of areas, ratio of lengths on collinear or parallel lines (eg. midpoints)
similarity 4 DOF	$\begin{bmatrix} r_{11} & r_{12} & T_x \\ r_{21} & r_{22} & T_y \\ 0 & 0 & s \end{bmatrix}$		all the above, plus ratio of lengths, angle
Euclidean 3 DOF	$\begin{bmatrix} r_{11} & r_{12} & T_x \\ r_{21} & r_{22} & T_y \\ 0 & 0 & 1 \end{bmatrix}$		all the above, plus length, area

Camera calibration: The process of determining the projection matrix P. Each point (u_i, v_i) gives a pair of equations, so at least 6 points are needed to calibrate the camera. The equations can be solve by linear least squares, and the solution can be refined by non linear minimisation of errors (between measured and projected points):

$$\min_{\mathrm{P}} \sum_i ((u_i - \hat{u}_i)^2 + (v_i - \hat{v}_i)^2)$$

Once P is known, decompose the 3×3 submatrix into $\mathbf{K}$ and R using RQ decomposition, and $\mathbf{T}$ can be found by $\mathbf{T} = \mathbf{K}^{-1} \begin{bmatrix} p_{14} & p_{24} & p_{34} \end{bmatrix}^\top$.

To calibrate the camera for viewing a plane, at least 4 points are needed, and for viewing a line, at least 3 points are needed.

Image mosaicing: Any two images of a general scene with the same camera centre (rotation only) are related by

$$\tilde{\mathbf{w}}' = \mathbf{K}\mathrm{R}\mathbf{K}^{-1}\tilde{\mathbf{w}}$$

A minimum of 4 image correspondences can be used to estimate the homography and stitch images together. The RANSAC algorithm can be used for robust fitting of the homography in presence of outliers.

Weak perspective projection: When the depth of objects in the scene is small compared to their distance from the camera, the projection can be approximated as a parallel projection, with a weak prosepective projection matrix

$$\mathrm{P}_{pll} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 0 & Z_c^{av} \end{bmatrix}$$

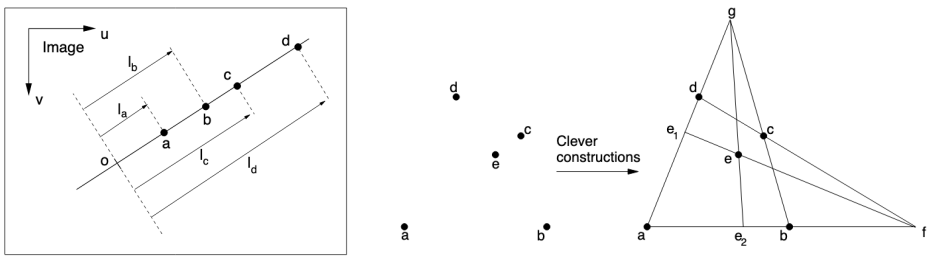
Affine camera: The affine camera generalises the weak perspective projection.

$$\mathrm{P}_{aff} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The 3D affine camera can be calibrated by at least 4 points, a 2D one can be calibrated by at least 3 points, and a 1D one can be calibrated by at least 2 points.

Cross-ratio: For 4 collinear points a, b, c and d, the cross-ratio is defined as

$$\frac{(X_d - X_a)(X_c - X_b)}{(X_c - X_a)(X_d - X_b)}$$

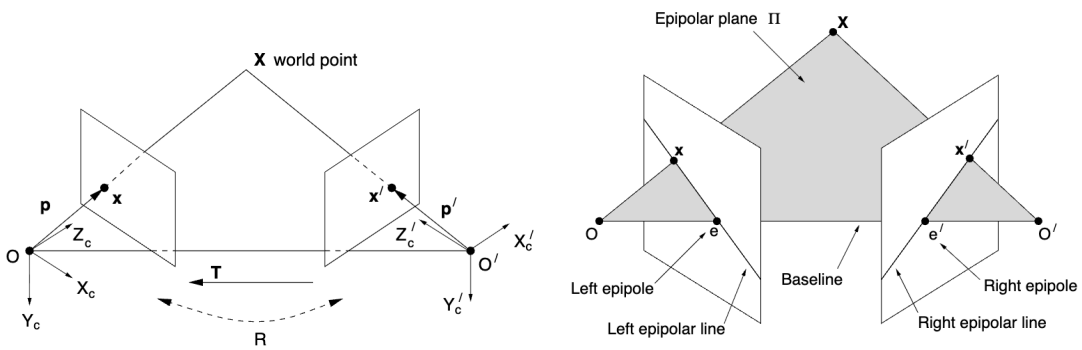


The cross-ratio is invariant under projection. 5 points in a plane can be used to construct two cross-ratios.

3 Stereo Vision

Rays: A ray is define by the point it pierces the image plane

$$\mathbf{p} = \begin{bmatrix} x \\ y \\ f \end{bmatrix} \quad \text{and} \quad \tilde{\mathbf{w}} = \mathbf{K}\mathbf{p}$$



Epipolar constraint: Correspondences must lie on the epipolar lines: the image of the ray from the other camera's optical centre to the point $\mathbf{X}$.

The epipolar constraint can be expressed as

$$\mathbf{p}'^{\top}\mathrm{E}\mathbf{p} = 0$$

where E is the *essential matrix*, which can be decomposed into a vector product matrix of the translation $\mathrm{T}_\times$ and a rotation matrix R between the two cameras:

$$\mathrm{E} = \mathrm{T}_\times \mathrm{R} \quad \text{where} \quad \mathrm{T}_\times = \begin{bmatrix} 0 & -T_z & T_y \\ T_z & 0 & -T_x \\ -T_y & T_x & 0 \end{bmatrix}$$

The epipoles follow $\mathrm{E}\mathbf{p}_e = \mathbf{0}$ and $\mathrm{E}^{\top}\mathbf{p}'_e = \mathbf{0}$. Hence, the E have a non-zero null space and is non-invertible.

For parallel cameras, the epipolar lines are parallel and the epipoles are at infinity.

Fundamental matrix: Since $\mathbf{p} = \mathbf{K}^{-1}\tilde{\mathbf{w}}$, the fundamental matrix F can be defined as

$$\mathrm{F} = \mathbf{K}'^{-\top}\mathrm{E}\mathbf{K}^{-1} \quad \text{such that} \quad \tilde{\mathbf{w}}'^{\top}\mathrm{F}\tilde{\mathbf{w}} = 0$$

The epipolar lines follow $\tilde{\mathbf{l}}' = \mathrm{F}\tilde{\mathbf{w}}$ and $\tilde{\mathbf{l}} = \mathrm{F}^{\top}\tilde{\mathbf{w}}'$, and the epipoles follow $\mathrm{F}\tilde{\mathbf{w}}_e = \mathbf{0}$ and $\mathrm{F}^{\top}\tilde{\mathbf{w}}'_e = \mathbf{0}$.

Since the overall scale is arbitrary and the determinant is zero, the fundamental matrix has 7 degrees of freedom. The fundamental matrix can be estimated by at least 8 correspondences.

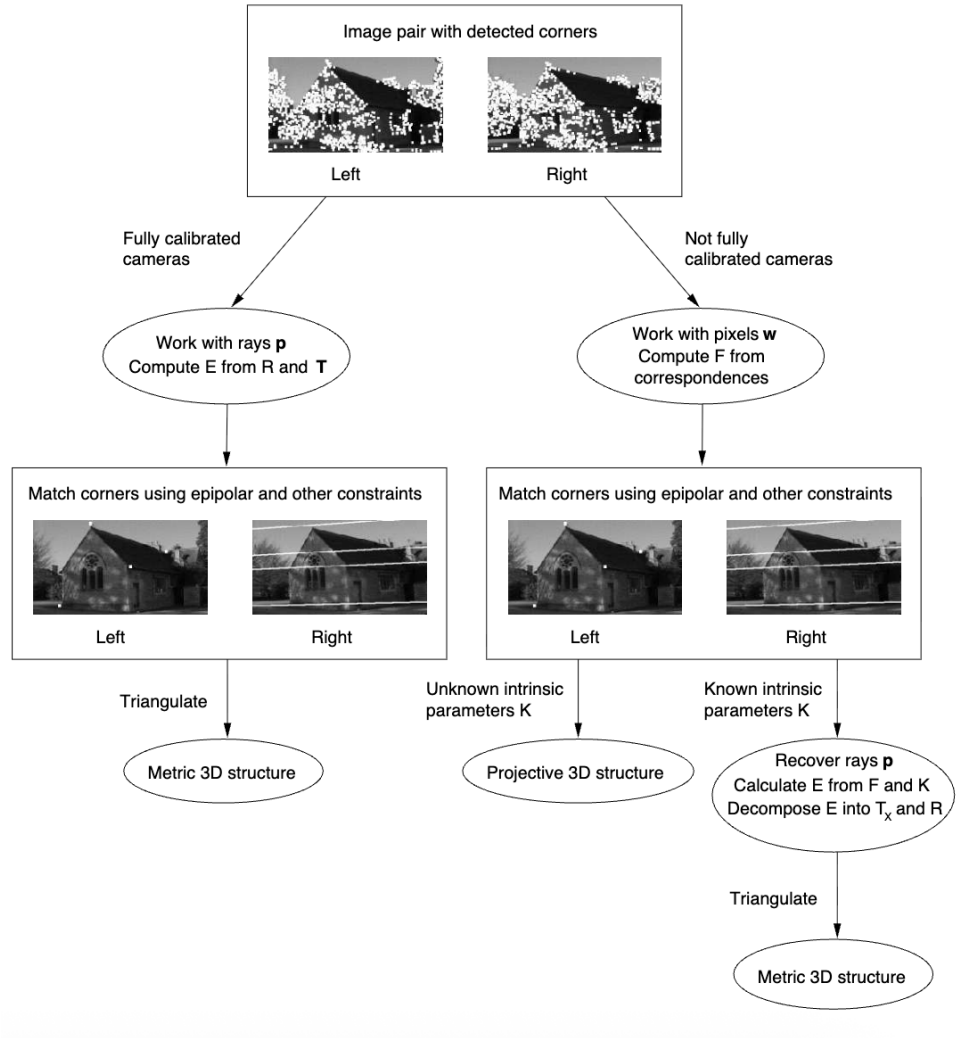
If the correspondences are noisy, the estimate will not have zero determinant – can be solved by setting the smallest singular value to zero.

Constraints on the correspondences:

1. *Uniqueness:* For opaque objects, each point in the left image has at most one match in the right image.
2. *Ordering:* For an opaque object, the points are ordered identically in two images.
3. *Figural continuity:* If the distinguished points lie on contours, they can be matched by the continuity of the contours.
4. *Disparity gradient:* If surfaces are smooth, the disparities (differences between locations of corresponding points) must be locally smooth.

Finding correspondences: The outline of an algorithm is:

1. Unguided matching: Use normalised cross-correalation to obtain a small number of seed matches.
2. Compute epipolar geometry: Use seed matches and robust regression to estimate the fundamental matrix F.
3. Guided matching: Restict the search for matches to a narrow band around epipolar lines.



Recovering metric structure: The singular value decomposition of the essential matrix gives

$$\mathrm{E} = \mathrm{U}\Lambda\mathrm{V}^\top = \mathrm{T}_\times\mathrm{R} \quad \text{where} \quad \mathrm{T}_\times = \mathrm{U}\begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}\mathrm{U}^\top \quad \text{and} \quad \mathrm{R} = \mathrm{U}\begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}\mathrm{V}^\top$$

which can be used to find the projection matrices:

$$\mathrm{P} = \mathbf{K}\begin{bmatrix}\mathrm{I} & \mathbf{0}\end{bmatrix} \quad \text{and} \quad \mathrm{P}' = \mathbf{K}'\begin{bmatrix}\mathrm{R} & \mathbf{T}\end{bmatrix}$$

The structure can then be recovered with triangulation (by least squares). The reconstruction is up to a scale factor. Ambiguity in $\mathbf{T}$ and R (4 pairs) can be resolved by checking the reconstructed points are in front of the cameras.

Affine stereo: When the depth of the scene is small compared to the distance from the camera, the affine camera is appropriate:

$$\begin{bmatrix} u \\ v \\ u' \\ v' \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p'_{11} & p'_{12} & p'_{13} & p'_{14} \\ p'_{21} & p'_{22} & p'_{23} & p'_{24} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Assuming the left camera is aligned with the world coordinate, the left projection matrix is $\begin{bmatrix} p_{11} & 0 & 0 & p_{14} \\ 0 & p_{22} & 0 & p_{24} \end{bmatrix}$. Substitution yields the epipolar constraint:

$$\mathbf{w}' = \mathbf{a} + Z\mathbf{b}$$

$\mathbf{b}$ is independent of $\mathbf{w}$ so all epipolar lines are parallel under affine stereo.

Affine fundamental matrix: The affine fundamental matrix is

$$\mathrm{F}_A = \begin{bmatrix} 0 & 0 & a \\ 0 & 0 & b \\ c & d & 1 \end{bmatrix} \quad \text{such that} \quad \tilde{\mathbf{w}}'^\top \mathrm{F}_A \tilde{\mathbf{w}} = 0$$

4 Deep Learning for Computer Vision

Note: Some of the contents are included in 4F10 hence not repeated here.

Perceptron: $y(\mathbf{x}) = f(\mathbf{w}^\top \mathbf{x} + b)$, where f is the activation function (e.g. $\tanh$).

Cross-entropy loss: $L = -\sum_i y_i \log \hat{y}_i$.

Batching: Use a subset of data to compute gradients and update weights and biases. It is known as *stochastic gradient descent* when the subsets are chosen at random.

Division of data:

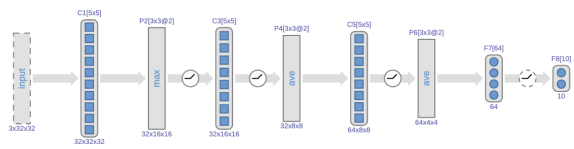
1. Training – fitting the model
2. Validation – determining the hyperparameters
3. Test – held out for evaluation

Rectified Linear Unit (ReLU): $f(x) = \max(0, x)$, used to solve the vanishing gradient problem of sigmoid and $\tanh$.

Convolution: Computing the dot product between a kernel and a local region of input (e.g. image patch).

Subsampling: Reducing the dimensionality of the convolution layer output by taking the maximum (max pooling) or average (average pooling) of a local region.

Convolutional Neural Networks (CNNs):



C1 Extracts low-level features in the image.
P2 Provides some flexibility of location
C3 Looks for parts that are combinations of features
P4 Smooths the part responses before subsampling
C5 Finds structures that are built from parts
P6 Smooths and subsamples the structural responses.
F7 Sub-category latent space
F8 Final classifier

Network in Network (NiN): Replace the fully connected layers with 1×1 convolution, which can be seen as a MLP applied across filter channels.

Semantic segmentation: Assign a class label to each pixel in the image. Replace the final classifier with an an interpolation layer.

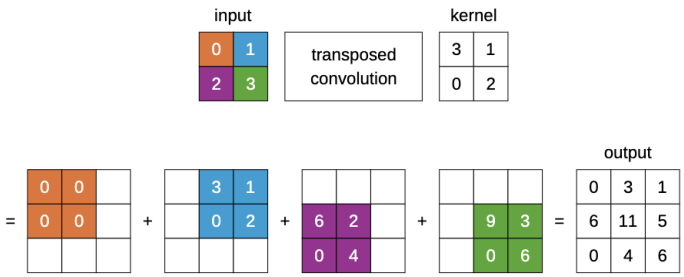
Autoencoding: Encoding the input into a latent space and decoding it back to the original input.

A *variational autoencoder* encodes the input into two vectors μ and σ and the system samples from $\mathcal{N}(\mu, \sigma)$ as the latent space input to the decoder. The loss function is the *evidence lower bound* (ELBO):

$$\mathcal{L} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{KL}(q(z|x)||p(z))$$

where the first term is the reconstruction loss (trains the generative decoder distribution to transform the latent vector back to input) and the second term is the regularisation loss (trains the encoder distribution $q(z|x)$ to be close to $p(z) = \mathcal{N}(0, I)$).

Transposed convolution: Increase the output dimension by scaling a kernel by an input pixel value and add it to a patch of the output image.



Instance segmentation (object detection): Assign multiple members of a class different labels.

You Only Look Once (YOLO): A network that divides the image into a grid of cells, and for each cell predict a set of bounding boxes and a class label for each box.

Feature embedding: Latent space representation such that similar inputs are close together and dissimilar inputs are larger. The *triplet loss* can be used for training:

$$\mathcal{L} = \sum_{i=1}^{N} \max(0, \|\mathbf{z}_a^i - \mathbf{z}_p^i\|_2^2 - \|\mathbf{z}_a^i - \mathbf{z}_n^i\|_2^2 + \alpha)$$

where the triplets are anchor point a, positive point p, and negative point n.

Positional encoding: Represent a function by a combination of sine and cosine functions of different frequencies.

Sampling from transformers: A couple of options:

- Sample from the output distribution, which can be adjusted by a temperature.
- Top-k sampling: Sample from the top k most probable tokens.
- Beam search: Keep track of the top k most probable sequences at each step and expand them in the next step. Select the most probable sequence at the end.